

Velociraptor DFIR + Wazuh

Integration on GCP

Complete Step-by-Step Guide for Beginners

Digital Forensics

Incident Response

Threat Hunting

Endpoint Investigation

Author: Samson Ejim

Cybersecurity Enthusiast

Google Cloud Platform | Ubuntu 22.04 LTS | Velociraptor 0.76.1 | Wazuh 4.7.5

Table of Contents

Introduction & Prerequisites	3
Architecture Overview	3
Detection Pipeline	4
Phase 1 — Install Velociraptor Server on wazuh-server VM	4
Phase 2 — Deploy Velociraptor Agent on ubuntu-endpoint	6
Phase 3 — Verify Agent Connection on Velociraptor Dashboard	8
Phase 4 — Run Your First Forensic Hunt	9
Phase 5 — Investigate a Simulated Incident	10
Understanding Your Forensic Artefacts	11
Troubleshooting Common Errors	12
Summary & Next Steps	13

Introduction

This guide walks you through deploying **Velociraptor 0.76.1** — a powerful open-source Digital Forensics and Incident Response (DFIR) platform — and integrating it with your existing Wazuh SIEM on Google Cloud Platform. While Wazuh detects threats via FIM and Suricata catches network intrusions, Velociraptor lets you **investigate what happened** after an alert fires. Together they form a complete SOC pipeline used in real enterprise environments.

What You Will Need

Requirement	Details
Existing Wazuh Lab	wazuh-server and ubuntu-endpoint VMs from previous labs still running
GCP Console Access	To SSH into both VMs and manage firewall rules
Suricata Lab Complete	This lab builds directly on top of the two previous projects
Basic Linux knowledge	Ability to run commands in a terminal

TIP

This lab reuses your existing GCP VMs — no new infrastructure costs!

Architecture Overview

Velociraptor runs in **server/client mode**. The Velociraptor Server runs on **wazuh-server** and exposes a web GUI on port 8889. The Velociraptor Agent runs on **ubuntu-endpoint** and connects back to the server over port 8000 using the **internal GCP IP (10.128.0.4)** — this never changes even when you restart your VMs, making it reliable for lab use.

VM	Internal IP	New Service	Ports
wazuh-server	10.128.0.4	velociraptor_server	8000 (agent comms), 8889 (GUI)
ubuntu-endpoint	10.128.0.3	velociraptor_client	Connects out to 10.128.0.4:8000

Detection Pipeline

The DFIR workflow with Velociraptor + Wazuh follows a logical investigate → hunt → respond chain:

1 Wazuh Alert	2 SOC Analyst Investigates	3 Velociraptor Hunt Launched	4 Forensic Artefacts	5 Evidence Collected
Rule 87105 or 86601	Alert triaged on dashboard	VQL query pushed to endpoint	Processes, files, network, timeline	Full incident report ready

PHASE 1

Install Velociraptor Server on wazuh-server VM

SSH into your **wazuh-server** VM from GCP Console. Click the SSH button next to wazuh-server in Compute Engine → VM Instances.

1 Open GCP Firewall Ports FIRST

Before installing, open the required ports in GCP Console → VPC Network → Firewall → Create Firewall Rule:

Field	Value
Name	velociraptor-ports
Direction	Ingress
Targets	All instances in network
Source IPv4 ranges	0.0.0.0/0
Protocols and ports	TCP: 8000, 8889
Priority	1000 (default — no need to change)

INFO

Port 8000 = agent communication. Port 8889 = web dashboard. Both are required.

2 Download Velociraptor Binary

The download URL structure is important — the **release tag** is **v0.76** but the **filename** includes **v0.76.1**. Using the wrong URL returns a 404 error.

```
# Download Velociraptor v0.76.1 for Linux amd64
# NOTE: The tag in the URL is v0.76 — NOT v0.76.1
wget
https://github.com/Velocidex/velociraptor/releases/download/v0.76/velociraptor-v0.76.1-linux-amd64

# Make executable and move to system path
chmod +x velociraptor-v0.76.1-linux-amd64
sudo mv velociraptor-v0.76.1-linux-amd64 /usr/local/bin/velociraptor

# Verify installation
velociraptor version
# Expected output:
# name: velociraptor
# version: 0.76.1
```

3 Generate Server Configuration

In v0.76.1, the config is generated non-interactively and then edited manually. This is more reliable than the interactive wizard which does not save the file automatically.

```
# Create the config directory
sudo mkdir -p /etc/velociraptor

# Generate default config
sudo velociraptor config generate > /tmp/server.config.yaml

# Verify config was created (should show ~13K file)
ls -lh /tmp/server.config.yaml
```

4 Update Config — Set Internal IP and Bind Addresses

The generated config uses **localhost** as hostname and **127.0.0.1** as bind addresses. We need to update these so the agent on ubuntu-endpoint can reach the server via the internal GCP network. Run these three commands:

```
# Step 1: Set hostname to wazuh-server internal IP
sudo sed -i 's/hostname: localhost/hostname: 10.128.0.4/' /tmp/server.config.yaml

# Step 2: Change first 127.0.0.1 to 0.0.0.0 (GUI bind address)
sudo sed -i '0,/bind_address: 127.0.0.1/s/bind_address: 127.0.0.1/bind_address: 0.0.0.0/' /tmp/server.config.yaml

# Step 3: Change ALL remaining 127.0.0.1 to 0.0.0.0
sudo sed -i 's/bind_address: 127.0.0.1/bind_address: 0.0.0.0/g' /tmp/server.config.yaml

# Verify — all bind_address should be 0.0.0.0, hostname should be 10.128.0.4
grep -E "hostname|bind_address" /tmp/server.config.yaml

# Expected:
# bind_address: 0.0.0.0
# bind_address: 0.0.0.0
# hostname: 10.128.0.4
# bind_address: 0.0.0.0
# bind_address: 0.0.0.0
```

5 Move Config to Final Location

```
sudo cp /tmp/server.config.yaml /etc/velociraptor/server.config.yaml
sudo chmod 600 /etc/velociraptor/server.config.yaml

# Verify
sudo ls -lh /etc/velociraptor/

# Expected: -rw----- 1 root root 13K ... server.config.yaml
```

6 Install Velociraptor Server as a System Service

In v0.76.1, the **service install** command no longer exists. Instead, generate a .deb package and install it with dpkg:

```
# Generate the server .deb package
# NOTE: Do NOT use "service install" — that command was removed in v0.72+
sudo velociraptor --config /etc/velociraptor/server.config.yaml debian server

# Expected output (the [ERROR] line is harmless — ignore it):
# [ { "OSPath": "/home//velociraptor-server-0.76.1.amd64.deb" } ]

# Install the package
sudo dpkg -i ~/velociraptor-server-0.76.1.amd64.deb

# Verify service is running
sudo systemctl status velociraptor_server

# Expected: Active: active (running) ■
```

7 Fix Permissions and Create Admin User

```
# Fix data directory permissions (prevents recurring permission errors)
sudo chown -R velociraptor:velociraptor /var/tmp/velociraptor

# Create admin user — MUST run as velociraptor user, not root
sudo -u velociraptor velociraptor --config /etc/velociraptor/server.config.yaml \
user add admin --role administrator

# Enter a password when prompted — write it down, you cannot recover it

# Restart server to pick up new user
sudo systemctl restart velociraptor_server
sudo systemctl status velociraptor_server

# Expected: Active: active (running) ■
```

8 Access the Velociraptor Dashboard

Use the **External IP** of wazuh-server to access the dashboard from your browser. The internal IP (10.128.0.4) is not reachable from outside GCP. Get the current external IP by running:

```
# Get current external IP of wazuh-server
curl -s ifconfig.me

# Example output: 35.225.192.161
```

WARNING

Open your browser and go to: https://<EXTERNAL_IP>:8889 Your browser will warn about a self-signed certificate. Click "Advanced" then "Proceed anyway" — this is expected in lab environments. Login with username: admin and the password you set in Step 7.

Deploy Velociraptor Agent on ubuntu-endpoint

Open a new **SSH session** into your ubuntu-endpoint VM from GCP Console. Keep the wazuh-server session open — you will need both.

1 Download Velociraptor Binary on ubuntu-endpoint

```
# Same binary, same URL – Velociraptor uses one binary for both server and client roles
wget
https://github.com/Velocidex/velociraptor/releases/download/v0.76/velociraptor-v0.76.1-linux-amd64

chmod +x velociraptor-v0.76.1-linux-amd64
sudo mv velociraptor-v0.76.1-linux-amd64 /usr/local/bin/velociraptor

# Verify
velociraptor version
# Expected: version: 0.76.1 ■
```

2 Generate Client Config on wazuh-server

Switch back to your **wazuh-server** SSH session and run:

```
# Generate client config from server config
sudo -u velociraptor velociraptor --config /etc/velociraptor/server.config.yaml \
config client > /tmp/client.config.yaml

# Check the server URL – it will say localhost by default
grep -A 2 "server_urls" /tmp/client.config.yaml
# Output:
# server_urls:
# - https://localhost:8000/

# Fix it to use the internal IP so ubuntu-endpoint can reach wazuh-server
sudo sed -i 's|https://localhost:8000/|https://10.128.0.4:8000/|' /tmp/client.config.yaml

# Verify the fix
grep -A 2 "server_urls" /tmp/client.config.yaml
# Expected:
# server_urls:
# - https://10.128.0.4:8000/ ■
```

3 Copy Client Config to ubuntu-endpoint

GCP VMs use SSH key authentication — SCP with password will fail. Use this manual copy method instead:

```
# On wazuh-server – print the entire client config
cat /tmp/client.config.yaml

# Copy ALL the output to your clipboard

# Switch to ubuntu-endpoint SSH session
# Create the file and paste the content
sudo nano /tmp/client.config.yaml
# Paste content with Ctrl+Shift+V
# Save with Ctrl+X → Y → Enter

# Verify on ubuntu-endpoint
grep -A 2 "server_urls" /tmp/client.config.yaml
# Expected: - https://10.128.0.4:8000/ ■
```

WARNING

If you try "scp" between VMs you will get "Permission denied (publickey)" — this is expected on GCP. The manual copy method above always works.

4 Install Velociraptor Client as a Service

Run these commands on **ubuntu-endpoint**:

```
# Generate the client .deb package
sudo velociraptor --config /tmp/client.config.yaml debian client
# Expected: [ { "OSPath": "/home//velociraptor_client_0.76.1_amd64.deb" } ]

# Install the package
sudo dpkg -i ~/velociraptor_client_0.76.1_amd64.deb

# Verify the client service is running
sudo systemctl status velociraptor_client
# Expected: Active: active (running) ■
```

PHASE 3

Verify Agent Connection on Velociraptor Dashboard

Open the Velociraptor dashboard in your browser at **https://<EXTERNAL_IP>:8889**. Get the current external IP with: **curl -s ifconfig.me** on wazuh-server.

1 Search for Connected Clients

```
# In the Velociraptor Dashboard:
# 1. Click "Search Clients" in the left navigation bar
# 2. Type "all" in the search box and press Enter
# 3. You should see ubuntu-endpoint listed with a GREEN dot

# Green dot = Online and connected ■
# Grey dot = Offline or not yet connected
```

TIP

If ubuntu-endpoint does not appear after 60 seconds, restart the client service: **sudo systemctl restart velociraptor_client** — then refresh the dashboard.

2 Verify Full Stack is Running on ubuntu-endpoint

```
# Run on ubuntu-endpoint to confirm all three services are active
sudo systemctl status wazuh-agent suricata velociraptor_client

# All three should show: Active: active (running) ■
# wazuh-agent – host detection and FIM
# suricata – network intrusion detection
# velociraptor_client – DFIR investigation agent
```

3 Run First VQL Query from the Shell Tab

Click on **ubuntu-endpoint** in the dashboard → click the **Shell** tab → set the mode to **VQL** (not Bash or PowerShell) → run:

```
-- List all running processes on ubuntu-endpoint
SELECT Pid, Name, Exe, CommandLine FROM pslist()

# Click Launch – results show 100+ processes including:
# wazuh-agent, suricata, velociraptor_client, systemd, sshd

# IMPORTANT: Always use VQL mode in the Shell tab
# The file_finder() plugin does NOT work in Shell tab VQL mode
# Use glob() instead for file searches (see Phase 5)
```

PHASE 4

Run Your First Forensic Hunt

A **Hunt** in Velociraptor is a forensic task pushed to one or many endpoints. You select a built-in Artifact (a named VQL template), Velociraptor pushes it to the client, and results stream back to the dashboard.

How to Create and Run a Hunt

Step	Action
1	Click "Hunt Manager" icon in the left navigation bar
2	Click "+ New Hunt" button
3	Enter a Description (e.g. "Process Investigation")
4	Click "Select Artifacts" — search for the artifact name
5	Select the artifact and click through to "Launch"
6	Back in Hunt Manager — select the hunt and click the Play (▶) button
7	Click "Run them all!" in the confirmation dialog
8	Click the Hunt ID to see results → click "Results" tab

Hunt 1 — Process List (Linux.Sys.Pslist)

```
# Artifact: Linux.Sys.Pslist
# Results: All running processes with PID, Name, User, Exe, Hash
# The Hash column contains MD5, SHA1, SHA256 of every process binary
# In a real SOC investigation you would compare these against VirusTotal
# Expected: 118 rows — all processes running on ubuntu-endpoint ■
```

Key VQL Queries — Quick Reference

VQL Query (Shell Tab)	What It Collects
SELECT Pid, Name, Exe, CommandLine FROM pslist()	All running processes
SELECT Laddr, Raddr, Status, Pid, Name FROM netstat()	All open network connections
SELECT FullPath, Mtime, Atime, Size FROM glob(globs="/root/*")	Files in /root (use glob not file_finder)
SELECT * FROM users()	All local user accounts

WARNING

In the Shell tab, always use glob() for file searches — NOT file_finder(). The file_finder() plugin requires a different execution context and will return "Plugin file_finder not found" error in the Shell tab.

PHASE 5

Investigate a Simulated Incident

This phase simulates a real SOC investigation. We drop EICAR test malware on ubuntu-endpoint, let Wazuh detect and auto-delete it, then use Velociraptor to prove forensically that it was there — exactly as a Tier 2 SOC analyst would.

Step 1 — Drop EICAR Malware on ubuntu-endpoint

```
# On ubuntu-endpoint — drop EICAR test malware in /root
sudo curl -Lo /root/malware_sample.com https://secure.eicar.org/eicar.com.txt

# Check if file exists immediately
sudo ls -lh /root/malware_sample.com

# If you see: No such file or directory
# GREAT! Wazuh Active Response already auto-deleted it ■
# This proves your Project 1 pipeline is still working perfectly
```

Step 2 — Investigate File Timeline with Velociraptor

Even though Wazuh deleted the file, Velociraptor can show the filesystem timeline proving the file existed. In the **Shell tab (VQL mode)** on ubuntu-endpoint:

```
-- Show recent file activity in /root directory
-- NOTE: Use glob() NOT file_finder() in the Shell tab
SELECT FullPath, Mtime, Atime, Ctime, Size
FROM glob(globs="/root/*")
ORDER BY Mtime DESC

# Results show all files with timestamps — ordered by most recently modified
# The malware_sample.com will NOT appear (already deleted)
# But you will see other forensic artefacts from your lab work
```

Step 3 — Check Open Network Connections

```
-- Check all open network connections on ubuntu-endpoint
SELECT Laddr, Raddr, Status, Pid, Name FROM netstat()

# Results show all listening and established connections
# Port 22 — SSH (expected)
# Port 20202 — Wazuh agent comms (expected)
# Port 20201 ESTAB — Wazuh connected to manager (expected)
# No unknown outbound connections = endpoint is clean ■
```

Full Alert Chain — End-to-End Validation

Tool	Rule/Event	Level	Description
Suricata + Wazuh	86601	3	Network scan detected — Nmap from wazuh-server
Wazuh FIM	554	5	File /root/malware_sample.com added to system
Wazuh + VirusTotal	87105	12	VirusTotal: malware confirmed — 66/90 engines
Wazuh Active Response	—	—	remove-threat.sh auto-deleted the file
Velociraptor	VQL Hunt	—	File timeline + netstat collected as forensic evidence

Understanding Your Forensic Artefacts

Artifact Name	Category	What It Collects
Linux.Sys.Pslist	Process	Running processes — PID, PPID, user, cmdline, file hashes
Linux.Network.Netstat	Network	Open TCP/UDP connections and listening ports
Linux.Sys.Users	Accounts	Local user accounts and last login info
Linux.Sys.Crontab	Persistence	Cron jobs across all users
Linux.Search.FileFinder	Files	Search filesystem by glob pattern or hash (Hunt only)
Linux.Forensics.Timeline	Timeline	Filesystem activity timeline
Linux.Sys.BashHistory	Logs	Bash command history for all users
Generic.Client.Info	System	Hostname, IP, OS, Velociraptor version

MITRE ATT&CK; Mapping

Technique ID	Technique Name	Tactic	Velociraptor Artifact
T1057	Process Discovery	Discovery	Linux.Sys.Pslist
T1049	System Network Connections Discovery	Discovery	Linux.Network.Netstat
T1087	Account Discovery	Discovery	Linux.Sys.Users
T1053	Scheduled Task/Job	Persistence	Linux.Sys.Crontab
T1005	Data from Local System	Collection	Linux.Search.FileFinder
T1059	Command and Scripting Interpreter	Execution	Linux.Sys.BashHistory

Troubleshooting Common Errors

All errors below were encountered during the actual lab session on 4 May 2026 and resolved successfully.

Error	wget returns ERROR 404: Not Found
Cause	Wrong URL format — the GitHub release tag is "v0.76" but the filename is "v0.76.1". Using v0.76.1 in the URL path gives 404.
Fix	Use the correct URL: https://github.com/Velocidex/velociraptor/releases/download/v0.76/velociraptor-v0.76.1-linux-amd64 — note v0.76 in the path, v0.76.1 in the filename.
Error	velociraptor config generate --interactive does not save config file
Cause	In v0.76.1 the interactive wizard no longer auto-saves to /etc/velociraptor/ by default.
Fix	Use non-interactive generation instead: <code>sudo velociraptor config generate > /tmp/server.config.yaml</code> — then manually edit and copy to /etc/velociraptor/.
Error	"service install" returns: expected command but got "service"
Cause	The "service install" command was removed in Velociraptor v0.72+. This is a breaking change from older guides.
Fix	Use the debian package method instead: <code>sudo velociraptor --config ... debian server</code> — then install with <code>sudo dpkg -i ...</code>
Error	Dashboard shows "Connection Refused" or times out on internal IP
Cause	The internal IP (10.128.0.4) is not reachable from your local browser — it is only accessible within GCP network.
Fix	Use the External IP for browser access. Get it with: <code>curl -s ifconfig.me</code> on wazuh-server. Open: https://:8889
Error	user add: "should be running as velociraptor user but you are root"
Cause	Running user add with plain sudo runs as root, which is rejected.
Fix	Run as the velociraptor user: <code>sudo -u velociraptor velociraptor --config /etc/velociraptor/server.config.yaml user add admin --role administrator</code>
Error	Permission denied on /var/tmp/velociraptor/config/inventory.json.db
Cause	The /var/tmp/velociraptor directory is owned by root, not the velociraptor service user.
Fix	Fix ownership: <code>sudo chown -R velociraptor:velociraptor /var/tmp/velociraptor</code> — then restart: <code>sudo systemctl restart velociraptor_server</code>
Error	SCP between VMs fails: "Permission denied (publickey)"
Cause	GCP VMs use SSH key authentication. SCP requires the destination VM to trust the source VM SSH key, which is not set up by default.
Fix	Use the manual copy method: <code>cat /tmp/client.config.yaml</code> on wazuh-server, copy the output, then <code>sudo nano /tmp/client.config.yaml</code> on ubuntu-endpoint and paste.
Error	Shell tab VQL: "Plugin file_finder not found"
Cause	The file_finder() plugin requires a different execution scope and does not work in the Shell tab VQL mode.
Fix	Use glob() instead: <code>SELECT FullPath, Mtime, Size FROM glob(globs="/root/*")</code> — glob() works correctly in the Shell tab.

Summary & Next Steps

You have now deployed a **complete three-layer SOC detection and investigation pipeline** on Google Cloud Platform across three projects:

Layer	Tool	Coverage	Capability
Network	Suricata IDS 8.0.4 40,000+ ET Rules	Network traffic	Intrusion detection MITRE ATT&CK; mapping
Host	Wazuh FIM 4.7.5 VirusTotal API	Files & system	File change detection Auto malware removal
SIEM	Wazuh Manager 4.7.5	Centralised	Alert correlation Dashboard & reporting
DFIR	Velociraptor 0.76.1	Endpoint forensics	Process, file, network Forensic investigation

Portfolio Deliverables — Project 3

Deliverable	Status
GitHub repo: velociraptor-dfir-wazuh-lab	Create and publish
README.md with badges, architecture, full writeup	Write using this guide
Screenshots — dashboard, hunt results, VQL queries, netstat	Taken during lab ■
Config files — server.config.yaml, client.config.yaml (redacted)	Save from lab
PDF guide in docs/	This document ■
LinkedIn post	Schedule after GitHub repo published

TIP

Your SOC stack — Suricata + Wazuh + Velociraptor — now covers the complete detect → alert → investigate → respond cycle. This is the foundation of a Tier 1 / Tier 2 SOC analyst skill set. Excellent work, Samson! Next recommended project: Splunk SIEM Lab.